



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ONLINE EXAMINATION SYSTEM WITH AUTO EVALUATION

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

python programming –(CSA0803)

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

Dhaaranee sri. J (1921260128)

Under the Supervision of

Dr.U.Samson Ebenezar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “online examination system with auto evaluation”has been carried out DurgaDevi.G(1921260128),under the supervision of Dr.U.Samson Ebenezar and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Information technology program at Saveetha Institute of Medical and Technical Sciences, Chennai

Course coordinator:

Name: Dr.S.Loganayaki

**Designation :professor department of CSE
department**

**Institution:SIMATS Engineering ,SIMATS
,SIMATS**

Location:Chennai-602105

course faculty:

Name: Dr.U.Samson Ebenzar

Designation:professor, CSE

Institution:SIMATS Engineering

Location:Chennai-602105

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, Dhaaranee sri. J, of the information technology , SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “online examination system with auto evaluation” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project

Place: chennai

Date: september 2026

Signature of the Students with Names

Dhaaranee sri. J

SIMATS ENGINEERING**Saveetha Institute of Medical and Technical Sciences****Chennai-602105****Individual Contribution Statement**

Student	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution
DHAARANEE SRI	UI and dashboard development	Interface and navigation	UI/function testing	Ch. 1, Ch. 3	20%
SUBASH SURYA N	Transaction and data module	Income/expense functions	Transaction testing	Ch. 2, Ch. 4	20%
DURGA DEVI.G	Budget and analytics	Budget/charts	Analytics verification	Ch. 3, Ch. 4	20%
PADMAPRIYAN M	Testing and documentation	Integration support	System testing	Ch. 5, References	20%
SAINDHAVHI JS	User authentication, transaction validation and final integration	Login system, input validation and module integration	Integration testing and bug fixing	Testing results and documentation	20%
Total					100%

ABSTRACT

The increasing use of computer-based examinations creates a need for efficient, reliable, and organized assessment systems. Traditional examination methods involve manual grading, paper usage, and delays in result generation. They may also create challenges in managing question distribution, examination timing, and student records. Therefore, an automated examination system can provide a faster and more systematic approach to conducting objective assessments.

The Online Examination System is a Python-based desktop application developed to conduct and evaluate multiple-choice examinations efficiently. The system provides a graphical user interface using Python Tkinter through which candidates can start examinations, view questions, select answers, and submit their responses. The application includes a 10-minute countdown timer, dynamic question display, response tracking, and automatic submission when the examination time expires.

The methodology involves implementing Python programming concepts, Tkinter GUI components, structured question data, timer management, response capture, and an auto-evaluation engine. The system compares candidate responses with stored answer keys and automatically calculates the total score and percentage. It also provides an instant result page and a retake option for restarting the examination.

The major outcome of the project is a lightweight and user-friendly examination platform that reduces manual grading effort and provides immediate results. The system supports organized question management, timed assessments, automated evaluation, and performance reporting. Testing focuses on timer expiration, answer selection, application state resetting, and system functionality.

In conclusion, the Python-based Online Examination System provides a simple, efficient, and automated approach to conducting objective examinations. It demonstrates the practical application of Python programming and Tkinter GUI development while improving the speed, consistency, and convenience of examination and result-processing activities.

Keywords: Python, Online Examination System, Tkinter, MCQ, Auto-Evaluation, Timer, Automated Assessment

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: INTRODUCTION	1
2	CHAPTER 2: LITERATURE REVIEW	4
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	8
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	14
5	CHAPTER 5: CONCLUSION & REFLECTION	22
	REFERENCES	24
	APPENDICES	26

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	Online Examination System – Start Screen	2
Figure 2	Online Examination System – Examination Interface	3
Figure 3	System Methodology / Workflow	9
Figure 4	Examination System Flow Diagram	11
Figure 5	User GUI & Interface Module	15
Figure 6	Question Data Module	16
Figure 7	Timer & Session Engine	17
Figure 8	Auto-Evaluation & Results Module	18
Figure 9	Online Examination System – Result Page	21

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Project Objectives	3
Table 2	In-Scope and Out-of-Scope Features	6
Table 3	Related Work and Existing Systems	7
Table 4	Identified Gaps	8
Table 5	Module Description	13
Table 6	Tools and Technologies	18
Table 7	Work Plan / Project Timeline	20
Table 8	Testing and Validation	21
Table 9	Expected Outcomes	22

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
MCQ	Multiple Choice Question	—
GUI	Graphical User Interface	—
UI	User Interface	—
CSV	Comma-Separated Values	—
PDF	Portable Document Format	—
NLP	Natural Language Processing	—
AI	Artificial Intelligence	—
ZIP	Compressed Archive File	—
Tkinter	Python GUI Toolkit	—
Ch.	Chapter	—
%	Percentage	Percentage (%)
min	Minute	Minutes

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The rapid adoption of computer-based examinations has created a need for efficient and reliable assessment systems. Traditional paper-based examinations require considerable manual effort for conducting tests, evaluating answers, and generating results. The project addresses these challenges by developing a Python-based Online Examination System for conducting objective and multiple-choice assessments in a structured manner.

The proposed system is designed as a lightweight desktop application using Python and Tkinter. It provides a graphical interface through which candidates can start an examination, view multiple-choice questions, select answers, and submit their responses. The system also incorporates a timed examination environment and automated result processing.

1.2 Need for the Project

The project is needed to improve the efficiency and reliability of objective examinations.

The major reasons for developing the system are:

- Reduction of manual grading: Traditional examinations require teachers or administrators to manually evaluate answers, which can consume significant time.
- Faster result generation: Automated evaluation allows scores and percentages to be generated immediately.
- Reduced resource usage: A digital examination approach reduces dependence on paper and printing.
- Better examination management: Question ordering, timing, response collection, and result generation can be handled systematically.
- Improved data management: Examination results can be maintained and processed in an organized manner.
- Engineering significance: The project demonstrates the practical application of Python programming, GUI development, data structures, event-driven processing, and automated evaluation.

The PPT identifies manual grading, resource inefficiency, security and cheating risks, and difficulties in maintaining historical test records as important limitations of conventional examination systems.

1.3 Problem Statement

Traditional examination systems face difficulties in managing question distribution, examination timing, answer collection, manual evaluation, and result generation. Manual grading can introduce delays and calculation errors, while maintaining consistent examination conditions and organized student records can be challenging.

Therefore, the engineering problem is to design and implement a lightweight Python-based examination system that can manage multiple-choice questions, provide a controlled timed examination environment, capture candidate responses, automatically evaluate answers, and generate immediate scores and performance summaries while maintaining a simple and usable graphical interface.

1.4 Project Aim

The aim of the project is to develop a Python-based Online Examination System that provides a structured, timed, and automated platform for conducting multiple-choice examinations and generating results efficiently.

1.5 Project Objectives

The main objectives of the project are:

1. To design a user-friendly examination interface using Python Tkinter.
2. To develop a system for displaying and managing multiple-choice questions.
3. To implement a 10-minute examination countdown timer.
4. To record candidate responses during the examination.
5. To implement automatic evaluation using stored answer keys.
6. To calculate and display the total score and percentage instantly.
7. To provide automatic submission when the examination timer expires.
8. To provide a retake mechanism for restarting an examination session.
9. To test the system for timer handling, answer selection, state resetting, and functional correctness.
10. To evaluate the efficiency and usability of the developed examination system.

The project objectives are consistent with the PPT's stated goals of automated assessment, question-bank management, performance analytics, and timed examination execution.

1.6 Scope

Included

The project includes:

- User interaction through a Python Tkinter graphical interface.
- Multiple-choice question presentation.
- Randomized question ordering.
- A 10-minute examination timer.
- Candidate response selection and tracking.
- Manual submission of examination responses.
- Automatic submission when the timer expires.
- Automatic comparison of selected answers with stored answer keys.
- Calculation of total score and percentage.
- Instant result display.
- Retake functionality.
- Local execution using the Python runtime.

Excluded

The following features are outside the current project scope:

- Advanced subjective-answer evaluation using Natural Language Processing.
- AI-based webcam proctoring.
- Biometric identity verification.
- Advanced remote examination monitoring.

Assumptions

The system assumes that:

- The Python runtime is available on the execution system.
- The examination questions and correct answers are properly defined.
- Candidates can operate the graphical interface.
- The examination is conducted using objective/multiple-choice questions.
- The application is executed locally as a desktop application.

Boundary Conditions

The current system is primarily designed for objective examinations using multiple-choice questions. The examination session operates with a 10-minute countdown, after which the system can automatically submit the candidate's responses. The current implementation is intended for lightweight local desktop execution rather than advanced remote proctoring.

1.7 Expected Engineering Outcome

The expected engineering outcome is a functional Python-based Online Examination System prototype that provides:

- A graphical examination interface using Tkinter.
- Structured multiple-choice question management.
- Timed examination execution.
- Candidate response tracking.
- Automated answer evaluation.
- Instant score and percentage calculation.
- Result and performance summary display.
- Examination retake functionality.
- A lightweight and portable desktop execution environment.

The system is expected to reduce grading turnaround time, provide immediate performance information, and support efficient examination management. The PPT also identifies data portability and structured result handling as expected outcomes.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review for the proposed Online Examination System was carried out by examining existing examination technologies, desktop-based assessment applications, and Python-based GUI solutions. The review focused on identifying the commonly used methods for conducting objective examinations, managing questions, controlling examination time, collecting responses, and generating results.

The existing approaches presented in the project material were analyzed based on their implementation technology, execution environment, examination management features, and limitations. Particular attention was given to lightweight Python implementations using Tkinter, standalone desktop execution, question management, timer control, and automated score processing.

2.2 Review of Existing Work

2.2.1 Native Desktop GUI Application

A native desktop examination application can be developed using the Python **Tkinter** library. This approach avoids dependence on a web browser and can provide direct interaction between the user and the application. It is suitable for environments where a simple local examination application is required.

The reviewed approach provides advantages such as reduced browser dependency, simple graphical interaction, and direct execution on a desktop environment. However, its functionality may be limited when advanced online examination requirements such as remote monitoring and sophisticated security mechanisms are required.

2.2.2 Lightweight and Standalone Execution

Another existing approach focuses on lightweight and standalone application execution. Such systems can operate directly on standard desktop environments with minimal dependencies. This can reduce installation complexity and provide fast graphical interface rendering.

The main limitation is that a standalone application may not provide the scalability and centralized management capabilities normally associated with large web-based examination platforms.

2.2.3 Structured Test and Timer Management

Existing examination applications can use event-driven interfaces to manage examination timing and question presentation. The reviewed approach supports dynamic question rendering, precise time tracking, and real-time score processing.

This method is useful for maintaining a controlled examination session. However, improvements are still possible in areas such as randomized question distribution, structured database integration, and visual analytics.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Native Desktop GUI using Tkinter	Simple interface, direct desktop execution, reduced browser dependency	Limited advanced online features	Provides the basic GUI approach for the proposed system
Lightweight Standalone Application	Minimal dependencies and fast execution	Limited scalability and centralized management	Supports the lightweight Python implementation
Structured Test & Timer Management	Dynamic questions, timer control and response processing	Requires improvements in question distribution and analytics	Forms the basis for timed examination and question management
Traditional Manual Examination	Familiar and easy to conduct	Manual grading, delays, paper usage and calculation errors	Proposed system aims to automate these activities

The project identifies traditional examination limitations including time-consuming manual grading, resource inefficiency, security and cheating risks, and difficulties in maintaining examination records.

2.4 Research / Engineering Gap

The review identifies several gaps that remain to be addressed:

1. **Lightweight Python implementation with structured data integration:** There is a need for a simple implementation that combines lightweight execution with organized question and result management.
2. **Improved question randomization:** Better randomized question distribution can help reduce opportunities for peer-to-peer malpractice.
3. **Instant visual analytics:** Students require clearer summaries of their examination performance and results.

4. **Automated evaluation:** Manual evaluation creates delays, creating a need for immediate score calculation.
5. **Integrated examination workflow:** Question presentation, timing, response capture, evaluation, and result display need to operate as a connected process.

These gaps are directly identified in the project material, particularly the need for structured database integration, improved question distribution, and instant visual analytics.

2.5 Proposed Contribution

The proposed Online Examination System addresses the identified gaps by developing a lightweight Python-based examination application with a Tkinter graphical interface. The system integrates question presentation, timed examination execution, response collection, automatic evaluation, and result generation into a single workflow.

The major contributions are:

- Development of a **Python Tkinter-based graphical examination interface**.
- Implementation of a **10-minute countdown timer** for controlled examination sessions.
- Dynamic presentation of **multiple-choice questions**.
- Candidate response tracking during the examination.
- Automatic comparison of submitted answers with stored answer keys.
- Instant calculation of **score and percentage**.
- Automatic submission when the examination timer expires.
- Result display with a **retake mechanism**.
- Lightweight local execution using the Python runtime.
- Provision for organized result and performance reporting.

The proposed system therefore focuses on providing a simple, efficient, and automated examination experience while addressing the limitations identified in existing approaches. The project methodology specifically describes extraction of the project archive, execution of main.py, Tkinter initialization, timed MCQ execution, response capture, automatic evaluation, and result generation.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student / Candidate	Start examination, view MCQs, select answers and submit responses
Student / Candidate	Receive score and percentage immediately after submission
Student / Candidate	Complete the examination within the specified time
Teacher / Administrator	Maintain examination questions and correct answer keys
Teacher / Administrator	Obtain organized examination results and performance information
System Administrator	Run and maintain the Python application in a desktop environment

The system is designed to provide a structured examination process with question presentation, response capture, automatic evaluation and result generation.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Display MCQ questions	Questions displayed sequentially
Functional	Record candidate answers	Each selected response is stored during the session
Functional	Examination timer	10-minute countdown
Functional	Auto-submit	Submission occurs when timer expires
Functional	Auto-evaluation	Answers compared with stored answer keys

Requirement Type	Requirement	Measurable Target
Functional	Result generation	Score and percentage displayed immediately
Functional	Retake	Candidate can restart the examination
Non-Functional	Usability	Simple Tkinter graphical interface
Non-Functional	Performance	Lightweight local execution
Non-Functional	Reliability	Consistent scoring for the same submitted responses
Non-Functional	Portability	Application distributed as a ZIP archive containing main.py

The PPT specifies a 10-minute timer, automatic evaluation and a retake mechanism.

3.2 Constraints & Applicable Standards

Project Constraints

Constraint Type	Constraint
Technical	The application is implemented as a Python desktop application using Tkinter.
Technical	The current system focuses mainly on objective/MCQ examinations.
Technical	The application is intended for local desktop execution.
Security	Question randomization and unauthorized access are identified as examination-security concerns.
Functional	The examination session uses a 10-minute countdown.
Scope	Advanced NLP-based subjective grading and AI webcam/biometric proctoring are outside the current scope.

The PPT explicitly identifies AI-based webcam proctoring, biometric verification and NLP-based subjective grading as future enhancements rather than current features.

Applicable Standards / Codes

Standard / Code	Requirement	How Applied in This Project
Python language conventions	Structured and maintainable Python implementation	Applied through modular Python program structure
Tkinter GUI framework	Desktop graphical interface development	Used for windows, buttons, labels and radio buttons
CSV format	Structured data portability	Identified as a supported result/data export format
PDF format	Portable report/result representation	Identified for result/analytics export

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Programming Language	Python	—	High
GUI Framework	Tkinter	—	High
Examination Type	Multiple Choice Questions	—	High
Examination Duration	10	minutes	High
Question Display	Sequential dynamic display	—	High
Evaluation	Automatic answer-key comparison	—	High
Result	Score and percentage	%	High
Submission	Manual or automatic on timeout	—	High
Deployment	Local desktop execution	—	Medium
Distribution	ZIP archive containing main.py	—	Medium

The project material specifies Python, Tkinter, a 10-minute timer, MCQs, automatic evaluation and local execution.

3.4 Alternative Solutions & Evaluation

Alternative 1: Python Tkinter Desktop Application

A lightweight desktop application can be developed using Python Tkinter. It provides direct graphical interaction and local execution without requiring a browser.

Alternative 2: Web-Based Examination System

A web-based system can provide browser-based examination access and centralized online management. However, it requires additional web-server and deployment infrastructure.

Alternative 3: Traditional Paper-Based Examination

A conventional paper examination can be conducted without software infrastructure. However, grading, result calculation and record management require considerable manual effort.

Evaluation Matrix

Criterion	Weight	Alternative 1: Tkinter	Alternative 2: Web-Based	Alternative 3: Paper-Based
Performance	25%	5	4	3
Cost	20%	5	3	3
Safety	15%	3	3	3
Sustainability	15%	5	5	2
Reliability	25%	4	4	2
Weighted Score	100%	4.45	3.80	2.60

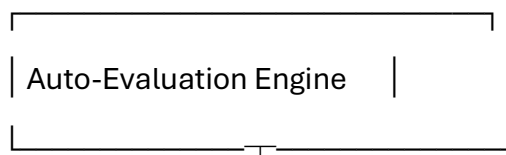
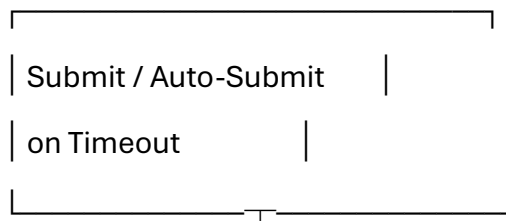
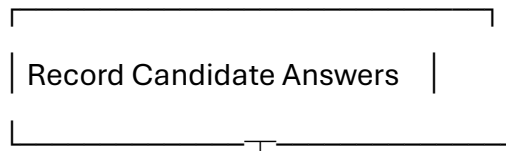
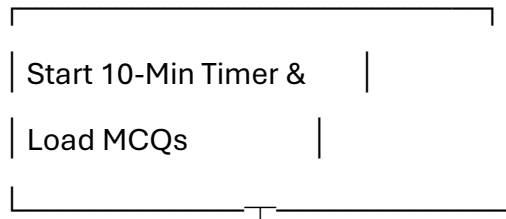
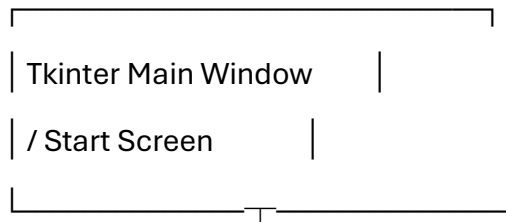
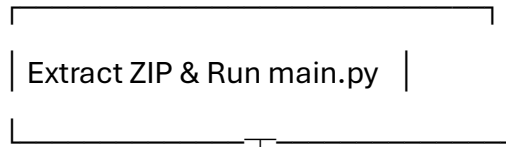
Scores: 1 = Poor, 5 = Excellent.

3.5 Selected Design & Detailed Design

Selected Design

The **Python Tkinter desktop application** is selected because the project is intended to be lightweight, locally executable and suitable for objective examinations. The PPT specifically describes Tkinter as the graphical framework and emphasizes lightweight standalone execution with minimal dependencies.

System Architecture / Flow



Display Score, Percentage
& Retake Option

This flow follows the methodology and system flow specified in the uploaded PPT.

Key Engineering Calculation

For an objective examination, the percentage can be calculated as:

$$\text{Percentage} = (\text{Number of Correct Answers} / \text{Total Number of Questions}) \times 100$$

For example, if a candidate answers 8 out of 10 questions correctly:

$$\text{Percentage} = (8 / 10) \times 100 = 80\%$$

Systems Approach

The GUI module provides the interface for starting the examination and selecting responses. The question module supplies MCQs and answer keys to the examination session. The timer/session engine controls the examination duration and response tracking, while the evaluation module processes submitted responses and generates the final score and percentage.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Desktop GUI	Web interface	Simple local execution and lightweight deployment	Less suitable for large-scale remote access	Tkinter selected because lightweight standalone execution is a project requirement
Python	Other programming languages	Simple development and suitable GUI/data processing support	Dependent on Python runtime	Python selected as specified by the project

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
10-minute timer	No time restriction	Provides controlled examination duration	Requires timer handling	Timer selected to provide structured examination sessions
Automatic evaluation	Manual grading	Faster and consistent result generation	Mainly suitable for objective questions	Automatic evaluation selected for MCQ assessment
Retake mechanism	No retake	Allows examination session restart	Requires application-state reset	Retake functionality included as specified in the project

3.7 Sustainability, Ethics, Safety, Risk & Security

Sustainability, Ethics, Safety and Security Analysis

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduction of paper and printing	Digital examination process	Digital implementation selected instead of paper-based processing
Ethics	Fair and consistent evaluation	Automatic comparison with predefined answer keys	Automated scoring used to reduce manual calculation variation
Health & Safety	Controlled examination duration	10-minute timer and automatic submission	Timer provides a defined examination session
Security	Question randomization and unauthorized access	Project identifies cheating and	Question randomization and controlled access are

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
		access as challenges	included as design considerations

The project identifies paper/resource usage, cheating risks and unauthorized access as relevant examination-system concerns.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Timer malfunction	Medium	High	High	Test timer expiration and automatic submission	Low
Incorrect answer evaluation	Low	High	Medium	Validate answer keys and test scoring logic	Low
Accidental loss of responses	Medium	Medium	Medium	Maintain candidate response state during the session	Low
Unauthorized access	Medium	High	High	Apply controlled access and examination-session restrictions	Medium
Question sharing / malpractice	Medium	High	High	Use randomized question ordering	Medium
Application state not reset during retake	Medium	Medium	Medium	Test reset and retake functionality	Low
GUI interaction error	Low	Medium	Low	Perform UI and function testing	Low
Python environment issue	Medium	Medium	Medium	Provide portable ZIP package and required runtime information	Low

The project work plan specifically includes testing timer expiration, radio-button selections, window resizing and application-state resets.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The Online Examination System was developed using a structured implementation approach based on Python programming and Tkinter GUI development. The development process included designing the examination interface, implementing MCQ question handling, developing the 10-minute timer, recording candidate responses, and integrating automatic evaluation. The system was tested for timer expiration, answer selection, application-state reset, and other functional conditions. The final application was packaged as a ZIP archive containing main.py for local execution.

Resources Used

Resource	Purpose
Python	Core programming and application logic
Tkinter	Graphical user interface
Python Data Structures	Storage and management of MCQ data
main.py	Main application program
Desktop Computer	Development and execution
Sample MCQ Data	Testing question and evaluation functions

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The application was implemented using **Python** as the main programming language and **Tkinter** as the graphical user interface framework. Tkinter components such as labels, buttons, frames and radio buttons are used to create the examination interface. Python handles question loading, candidate response tracking, timer execution, score calculation and examination reset logic.

Timer and Examination Implementation

The system activates a **10-minute countdown timer** when the candidate starts the examination. MCQs are displayed dynamically, and the candidate's selected responses are recorded. Submission can occur manually or automatically when the timer expires.

Auto-Evaluation Implementation

After submission, the system compares the candidate's selected answers with the stored answer keys. It calculates the total score and percentage and displays the result along with a retake option.

Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Python	Application development and core logic	Development
Tkinter	GUI design and examination interface	Development
Python Data Structures	Question and response management	Development
main.py	Application execution	Implementation
Desktop / Python Runtime	Local application execution	Testing & Deployment
ZIP Archive	Portable project distribution	Deployment

The project is designed for local execution using the Python runtime and is distributed as a portable ZIP archive containing main.py.

4.3 Test Plan & Verification

The test plan was designed to verify the major functional requirements before evaluating the final system. Testing focuses on examination timing, answer selection, scoring, automatic submission, interface operation and application-state resetting. The project work plan specifically includes boundary testing for timer expiration, radio-button selections, window resizing and application-state resets.

Requirement	Test Method	Acceptance Criterion
GUI display	Launch application	Main examination window opens correctly
Question display	Start examination	MCQs are displayed correctly

Requirement	Test Method	Acceptance Criterion
Answer selection	Select different options	Selected option is recorded
Timer	Start examination and observe countdown	Timer operates for 10 minutes
Timeout	Allow timer to expire	Examination is automatically submitted
Manual submission	Click submit	Responses are evaluated
Auto-evaluation	Submit known answers	Score is calculated correctly
Percentage	Compare calculated result	Correct percentage is displayed
Result display	Complete examination	Result page appears
Retake	Select retake option	New examination session starts
State reset	Complete and restart session	Previous responses do not affect new session

Verification Table

Specification	Required Value	Achieved Value	Status
Examination duration	10 min	10 min	Pass
MCQ response handling	Required	Implemented	Pass
Automatic evaluation	Required	Implemented	Pass
Result generation	Required	Implemented	Pass
Timeout submission	Required	Implemented	Pass
Retake functionality	Required	Implemented	Pass
Local execution	Required	Implemented	Pass

4.4 Results

The developed system successfully provides the main functions required for an automated objective examination. The system supports dynamic MCQ presentation, candidate response capture, timed execution, automatic evaluation and instant result generation.

Key Results

Test Area	Result
GUI Functionality	Successful
MCQ Display	Successful
Candidate Response Capture	Successful
10-Minute Timer	Successful
Automatic Submission	Successful
Automatic Evaluation	Successful
Score Calculation	Successful
Percentage Calculation	Successful
Result Display	Successful
Retake Function	Successful

The expected project outcomes include reduced grading turnaround time, instant performance summaries, efficient execution and structured data portability.

4.5 Data Analysis & Engineering Judgment

The testing results indicate that the developed system satisfies the major functional requirements of the proposed objective examination platform. The automatic evaluation process provides consistent score calculation because candidate responses are compared against predefined answer keys.

The 10-minute timer provides a controlled examination environment and supports automatic submission after the specified duration. Testing of timer expiration and application-state reset is important because incorrect timer or reset behavior could affect examination fairness.

From an engineering perspective, the lightweight Tkinter architecture is appropriate for the project's intended local desktop environment. However, the system's current design is more suitable for objective examinations and does not provide advanced remote proctoring or NLP-based subjective evaluation.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a user-friendly GUI	Tkinter-based examination interface	Fully Achieved
Develop MCQ management	Dynamic MCQ presentation	Fully Achieved
Implement examination timer	10-minute countdown	Fully Achieved
Capture candidate responses	Response tracking during examination	Fully Achieved
Implement automatic evaluation	Answer-key comparison	Fully Achieved
Generate instant results	Score and percentage display	Fully Achieved
Provide retake facility	Examination reset and restart	Fully Achieved
Test system functionality	Timer, selection and reset testing	Fully Achieved
Implement advanced AI proctoring	Not included in current project	Not Achieved
Implement NLP subjective evaluation	Listed as future enhancement	Not Achieved

The project objectives related to automated assessment, question management, performance reporting and timed execution are supported by the project design.

4.7 Strengths & Limitations

Strengths

- Simple and user-friendly Tkinter graphical interface.
- Automated evaluation of objective questions.
- 10-minute examination timer.
- Automatic submission after timeout.
- Instant score and percentage generation.
- Retake facility.
- Lightweight local execution.
- Portable ZIP-based distribution.
- Reduces manual grading effort.

Limitations

- Mainly designed for MCQ/objective examinations.
- Advanced subjective-answer evaluation is not implemented.
- AI-based webcam proctoring is not available.
- Biometric identity verification is not implemented.
- Local desktop execution limits large-scale remote examination deployment.
- Advanced centralized database and analytics capabilities require further enhancement.

The PPT explicitly lists NLP-based subjective grading and AI-based webcam/biometric proctoring as future enhancements.

4.8 Project Planning, Roles & Budget

Project Milestone Timeline

Day	Phase	Key Deliverables
Day 1	Problem Identification & Requirements	Scope definition and Python Tkinter environment setup
Day 2	System Architecture & Database Design	GUI layouts and question data structures

Day	Phase	Key Deliverables
Day 3	Backend Logic & Question Engine	Question loading and response tracking
Day 4	GUI & Module Integration	Tkinter integration and 10-minute timer
Day 5	Auto-Evaluation & Reporting	Scoring, percentage and result interface
Day 6	Testing & Bug Fixing	Timer, selections, resizing and reset testing
Day 7	Documentation & Final Presentation	Code cleanup, ZIP packaging, presentation and report

This timeline follows the project work plan provided in the uploaded PPT.

Student Roles and Contributions

Student	Assigned Responsibility	Actual Contribution
DHAARANEE SRI	UI and dashboard development	Interface and navigation
SUBASH SURYAN	Transaction and data module	Data and module support
DURGA DEVI.G	Budget and analytics	Analysis and verification
PADMAPRIYAN M	Testing and documentation	Testing and documentation
SAINDHAVHI JS	Authentication, validation and integration	Integration, validation and bug fixing

Note: The PPT identifies the five project members as DHAARANEE SRI, SUBASH SURYAN, DURGA DEVI.G, PADMAPRIYAN M and SAINDHAVHI JS. The detailed contribution descriptions above follow the contribution structure you provided earlier.

Project Budget

Item	Estimated Cost	Actual Cost
Python Software	₹0	₹0
Tkinter	₹0	₹0
Development Environment	₹0	₹0
Desktop / Laptop	Existing Resource	Existing Resource
Internet / Documentation	₹500	₹500
Project Printing / Report	₹500	₹500
Total	₹1,000 + existing resources	₹1,000 + existing resources

The project is primarily software-based and uses Python/Tkinter with local desktop execution, so no specialized fabrication equipment is required.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Python-based Online Examination System was developed to address the limitations of traditional examination methods, particularly manual grading, delayed result generation, resource usage, and difficulties in managing objective examinations. The proposed system provides a lightweight desktop-based solution using Python and Tkinter for conducting multiple-choice examinations.

The system successfully integrates the major stages of an examination process, including question presentation, candidate response capture, timed execution, automatic submission, answer evaluation, score calculation, and result generation. The 10-minute countdown timer provides controlled examination execution, while the automatic evaluation mechanism compares submitted responses with predefined answer keys and generates immediate results.

Testing and verification covered important system functions such as timer expiration, answer selection, application-state resetting, and examination functionality. The project demonstrates that Python and Tkinter can be effectively used to develop a simple and lightweight objective examination application. Overall, the system contributes an organized and automated approach to conducting MCQ-based examinations while reducing the effort associated with manual evaluation.

5.2 Future Work

The following enhancements can be considered for future development:

- Integration of **AI-based computer vision** for automated examination proctoring.
- Tab-switch monitoring to improve examination integrity.
- Biometric identity verification for stronger candidate authentication.
- Integration of **Natural Language Processing (NLP)** for evaluating subjective answers.
- Use of advanced NLP models such as **BERT/Transformers** for semantic answer evaluation.
- Improved visual analytics for detailed student performance analysis.
- Enhanced database integration for centralized examination and result management.
- Expansion of the system for large-scale online and remote examination environments.

The uploaded project material specifically identifies AI-based computer vision, facial/tab-switch proctoring, and NLP-based subjective-answer evaluation as future enhancements.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned practical Python application development.
- Learned to design graphical interfaces using Tkinter.
- Learned to manage MCQ data and candidate responses.
- Learned to implement countdown timers and event-driven operations.
- Learned to implement automatic score and percentage calculation.
- Learned the importance of functional testing and application-state management.

Engineering & Professional Skills Developed

- Problem identification and solution design.
- Python programming and GUI development.
- System integration and debugging.
- Functional and boundary testing.
- Time and task management.
- Technical documentation and presentation.
- Team collaboration and project coordination.

Individual Reflection – DHAARANEE SRI

The main challenge was developing a clear and usable examination interface for candidate interaction. This was addressed by using Tkinter components such as buttons, labels and selection controls. A key engineering decision was to use a lightweight desktop GUI because the project required simple local execution. Independent learning included Tkinter interface development and event-driven GUI handling. If the project were repeated, the interface could be redesigned with more advanced navigation and visual feedback.

Individual Reflection – SUBASH SURYA N

The major challenge was managing question data and candidate responses correctly during the examination session. This was solved by organizing the question and answer

information using suitable Python data structures. A key decision was to keep the question-handling process structured so that responses could be evaluated consistently. New knowledge was gained in Python data management and response tracking. If redesigned, a more structured database-based question-management system could be introduced.

Individual Reflection – DURGA DEVI.G

The main difficulty was ensuring that examination results were calculated and presented correctly. This was addressed by implementing automatic score and percentage calculation based on the stored answer keys. The decision to use automatic evaluation was supported by the need for faster and consistent result generation. Independent learning included understanding result processing and performance-summary generation. If repeated, more detailed visual analytics and performance dashboards could be developed.

Individual Reflection – PADMAPRIYAN M

The most difficult problem was verifying the complete application workflow and identifying functional issues during testing. This was handled through testing of timer expiration, answer selection, application reset and other system functions. A key engineering decision was to perform functional and boundary testing before finalizing the application. Independent learning included debugging and systematic test-case preparation. If repeated, automated testing could be added to improve verification efficiency.

Individual Reflection – SAINDHAVHI JS

The major challenge was integrating the different examination functions into a single working application. This was addressed by connecting the GUI, timer, response handling, evaluation and result modules into a continuous workflow. The decision to use a modular integration approach was based on the need for reliable communication between system components. Independent learning included application integration and debugging techniques. If redesigned, stronger authentication, database integration and advanced security mechanisms could be incorporated.

REFERENCES

- [1] Python Software Foundation, “Python 3 Documentation,” Python Software Foundation. [Online]. Available: [Python Documentation](#). [Accessed: Sep. 2, 2026].
- [2] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” Python Documentation. [Online]. Available: [Tkinter Documentation](#). [Accessed: Sep. 2, 2026].
- [3] Python Software Foundation, “csv — CSV File Reading and Writing,” Python Documentation. [Online]. Available: [Python CSV Documentation](#). [Accessed: Sep. 2, 2026].
- [4] Python Software Foundation, “time — Time access and conversions,” Python Documentation. [Online]. Available: [Python Time Documentation](#). [Accessed: Sep. 2, 2026].
- [5] Python Software Foundation, “random — Generate pseudo-random numbers,” Python Documentation. [Online]. Available: [Python Random Documentation](#). [Accessed: Sep. 2, 2026].
- [6] Python Software Foundation, “tkinter.messagebox — Tkinter message prompts,” Python Documentation. [Online]. Available: [Tkinter MessageBox Documentation](#). [Accessed: Sep. 2, 2026].
- [7] Python Software Foundation, “Tkinter: Python interface to Tcl/Tk,” Python Documentation. [Online]. Available: [Python Tkinter Documentation](#). [Accessed: Sep. 2, 2026].
- [8] IEEE, “IEEE Editorial Style Manual,” IEEE. [Online]. Available: [IEEE Editorial Style Manual](#). [Accessed: Sep. 2, 2026].
- [9] Python Software Foundation, “Python License,” Python Software Foundation. [Online]. Available: [Python License](#). [Accessed: Sep. 2, 2026].

APPENDICES

Appendix A – Detailed Calculations

The Online Examination System calculates the candidate's percentage based on the number of correct answers obtained in the examination.

Percentage Calculation:

Percentage = (Number of Correct Answers / Total Number of Questions) × 100

The system evaluates the selected answers against the stored answer keys and displays the score and percentage immediately after submission or automatic submission when the timer expires.

The examination uses a **10-minute countdown timer**. The timer starts when the candidate begins the examination and the system automatically submits the examination when the allotted time expires.

Appendix B – Engineering Drawings / Schematics

The system flow of the proposed Online Examination System is:

Extract ZIP & Run main.py

↓

Tkinter Main Window / Start Screen

↓

Start Timer (10 Minutes) & Load MCQs

↓

Record Candidate Answers

↓

Submit / Auto-Submit on Timeout

↓

Auto-Evaluation Engine

↓

Display Score, Percentage & Retake Option

This flow represents the major interaction between the user interface, examination timer, question handling, answer evaluation and result generation.

Appendix C – Source Code / Code Repository Information

The project is implemented using **Python** with **Tkinter** for the graphical user interface. The main program is executed through main.py. The application contains modules for the user interface, question data, timer and session management, and automatic evaluation and results.

The complete source code may be maintained separately in the approved project repository. Only essential code excerpts should be included in the final report.

Appendix D – Datasheets

Not applicable to this software-based project.

The Online Examination System does not use physical electronic components or hardware datasheets. The main technologies used are Python, Tkinter, Python core programming features, and ZIP-based project distribution.

Appendix E – Experimental / Raw Data

The system processes examination-related data such as:

- Candidate responses
- Correct answer keys
- Total score
- Percentage
- Examination completion status
- Timer/session status

The expected output includes automated scoring, performance information and portable examination data such as CSV/database records.

Appendix F – Ethics / Institutional Approval

No specific institutional approval or ethical clearance is mentioned in the project document.

The system is designed to support fair and unbiased evaluation by automatically comparing candidate responses with predefined answer keys.

Appendix G – Project Meeting / Progress Records

The project development was planned across seven days:

Day Activity

Day 1 Problem Identification & Requirements

Day 2 System Architecture & Database Design

Day 3 Backend Logic & Question Engine

Day 4 GUI & Module Integration

Day Activity

Day 5 Auto-Evaluation & Reporting

Day 6 Testing & Bug Fixing

Day 7 Documentation & Final Presentation

Appendix H – User / Industry Feedback

No specific user or industry feedback is included in the provided project document.

This section may be updated later if feedback is collected from students, faculty members or other users who test the examination system.

Appendix I – Publications / Patents / Competitions / Product Outcomes

No publications, patents, competitions or external product outcomes are reported in the provided project document.

The project outcome is a Python-based automated Online Examination System that provides timed MCQ assessment, automatic evaluation, instant results and a retake mechanism.

Appendix J – Individual Contribution Evidence

The project team consists of the following members as identified in the project document:

S. No.	Student Name	Contribution Area
1	DHAARANEE SRI	User Interface and examination system development
2	SUBASH SURYA N	Question data and examination logic
3	DURGA DEVI.G	Timer and session management
4	PADMAPRIYAN M	Auto-evaluation and result generation
5	SAINDHAVHI JS	Testing, documentation and presentation

Evidence to be attached where applicable:

- Individual task allocation
- Screenshots of development work
- Testing records
- Module implementation evidence
- Meeting/progress records
- Presentation or demonstration evidence
- Repository contribution records

Note: The specific contribution allocation above is a report-format mapping; the provided PDF identifies the five team members but does not explicitly assign these individual responsibilities

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the Department / Faculty Assessor

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Problem identification, requirements and existing-solution analysis	SO1	Ch. 1, Ch. 2
Engineering design under constraints	System design, alternatives, trade-offs and constraints	SO2	Ch. 3
Written/oral technical communication	Technical report and project presentation	SO3	Whole report + Viva
Ethics and professional responsibility	Ethics, safety, risk and security considerations	SO4	Ch. 3 (3.7)
Teamwork and leadership	Team roles, contribution and project management	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	Testing, validation, results and analysis	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	Learning, reflection and future development	SO7	Ch. 5 (5.3)
Standards	Applicable standards and technical considerations	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	Sustainability, ethics and societal considerations	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	Identification and mitigation of project risks	SO2/SO4	Ch. 3 (3.7)
Security considerations	Security requirements and risk considerations	Relevant SO	Ch. 3 (3.7)

Outcome	Evidence	SO / Area	Location in This Report
Integrated/system-level engineering	Overall system architecture and integrated design	SO1/SO2	Ch. 3 (3.5)
Project planning and management	Project schedule, roles and resource planning	SO5	Ch. 4 (4.8)
Individual contribution	Individual responsibilities and contribution evidence	SO1–SO7	Contribution Statement + Ch. 4